



MSE 153: Introduction to IT

Files, Directories and File System

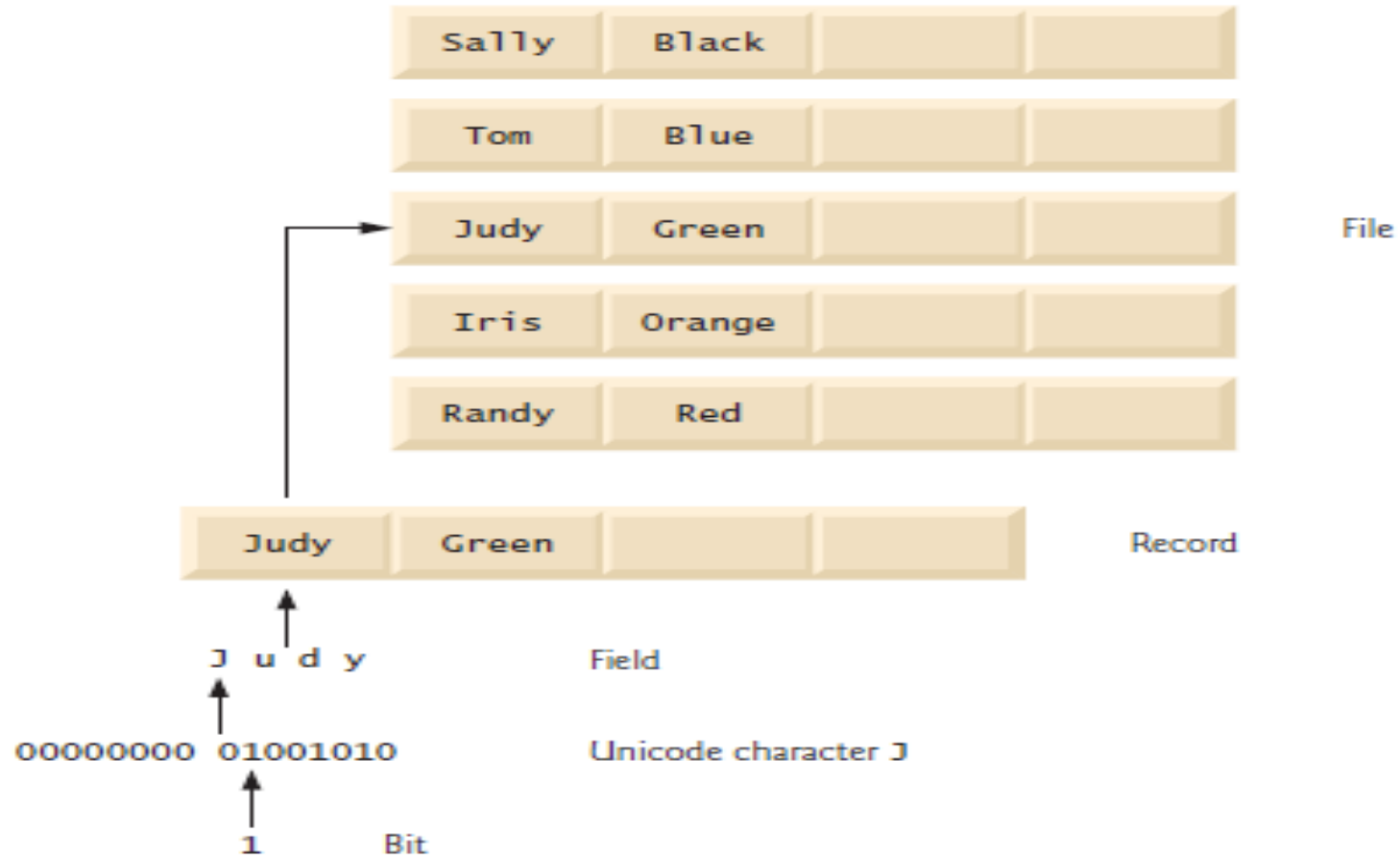


Lecture Outline

- Files and Directories
- File Systems and Disks
- Linux File System
- Windows File System
- Moving around the File System
 - Linux
 - DOS
- File System and System Administration Tasks



Data Hierarchy





Files, Directories, Partitions

In a file system, you have three types of entities: Partitions, Directories and Files.

- Files – executable (programs), data i.e. Files are the meaningful units of storage in a file system.
- Directories - store files and subdirectories
 - Creates a hierarchical file space used to organize the file space (/ in Linux, \ in Windows to denote directory)
- Directories are used to create logical divisions in the file system
- Partitions create physical divisions in the file system
 - Different directories reside in different partitions
 - Unmounting one partition does not impact others
 - Partitions are independent of each other
 - mount and unmount one without affecting others
 - back up one without impacting others
 - place disk quotas on one but not others



Files and File Operations

- Names / extensions
 - Types
 - Pointer(s) to physical storage of file
 - number of pointers (hard links)
 - Creation, modification, access dates
 - Owner, group
 - Permissions
- Linux/DOS
 - ls/dir – list directory contents
 - cp/copy – copy file
 - mv/move – move or rename file
 - rm/del – delete file
 - cat, more, less/type – display textfile contents
 - mkdir/mkdir – create directory
 - rmdir/rmdir – delete directory (empty dir only)
 - Windows (use Windows Explorer)
 - select directory from left hand pane to view its contents in the right hand pane
 - drag icons to copy or move
 - select icon and press <delete> (or drag to trash can) to delete
 - right click on icon and select rename
 - right click and select New → Folder to create directory

☆ Favorites
 Desktop
 Downloads
 Recent Places
 CLASSES

Libraries

Computer

Local Disk (C:)

- data
- db
- Dev-Cpp
- FPC
- Intel
- MSOCache
- PerfLogs
- Program Files
- Program Files (x86)
- ProgramData
- Python32
- Ruby192
- Users
- Windows
- VC_RED.cab

FreeAgent Drive (E:)

- foxr\$ (\\fileserv3.hh.nku.edu)
- Departments\$ (\\fileserv3.hh.nku.edu)
- foxr (\\sappho.hh.nku.edu)

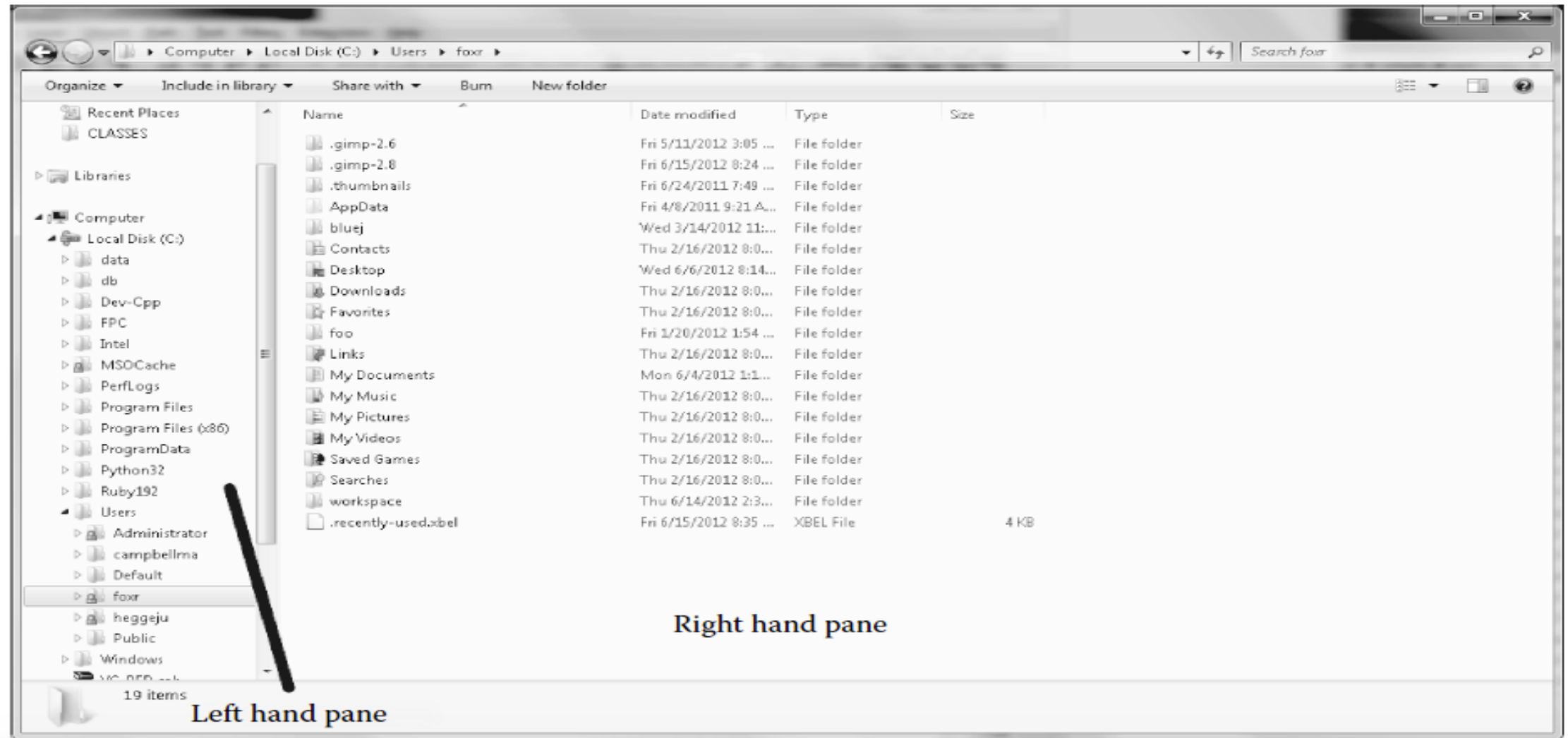
Network

File Edit View Places Help

Name ▼

- Floppy Drive
- CD-RW/DVD-RAM Drive: CentOS_5.6_Final
- FileSystem
- Network

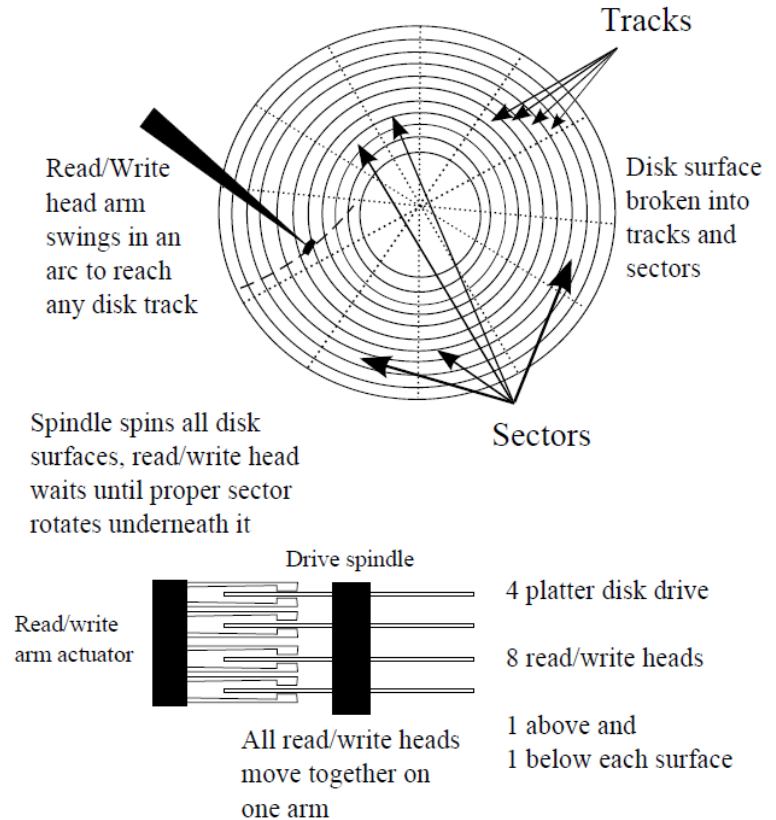
bin	110 items	folder	Tue 23 Aug 2011 12:51:38 PM EDT
boot	7 items	folder	Tue 23 Aug 2011 07:34:22 AM EDT
dev	192 items	folder	Tue 29 May 2012 08:08:03 AM EDT
etc	232 items	folder	Tue 29 May 2012 09:13:40 AM EDT
home	2 items	folder	Tue 23 Aug 2011 11:43:02 AM EDT
lib	145 items	folder	Tue 23 Aug 2011 12:51:01 PM EDT
lost+found	7 items	folder	Tue 23 Aug 2011 07:30:35 AM EDT
media	1 item	folder	Tue 29 May 2012 08:26:16 AM EDT
misc	0 items	folder	Tue 29 May 2012 08:07:59 AM EDT
mnt	0 items	folder	Tue 26 Jan 2010 04:43:49 PM EST
net	0 items	folder	Tue 29 May 2012 08:07:59 AM EDT
opt	0 items	folder	Tue 26 Jan 2010 04:43:49 PM EST
proc	177 items	folder	Tue 29 May 2012 08:06:58 AM EDT
root	7 items	folder	Tue 29 May 2012 08:29:30 AM EDT
sbin	277 items	folder	Tue 23 Aug 2011 12:51:38 PM EDT
selinux	17 items	folder	Tue 29 May 2012 08:06:59 AM EDT
srv	0 items	folder	Tue 26 Jan 2010 04:43:49 PM EST
sys	9 items	folder	Tue 29 May 2012 08:06:59 AM EDT
tmp	8 items	folder	Mon 04 Jun 2012 04:02:02 AM EDT
usr	13 items	folder	Tue 23 Aug 2011 07:32:01 AM EDT





File System Storage

- Mounting / unmounting
 - Adding/removing partitions from the file system
 - partitions may share same physical storage space (e.g., same drive)
 - partitions may be spread across multiple drives
- Disk blocks
 - Files broken into fixed size chunks, blocks

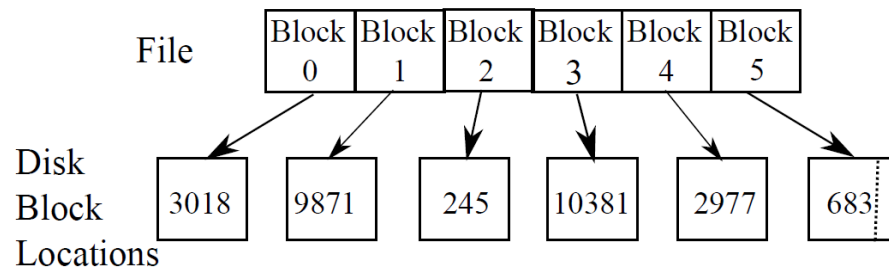


File's blocks distributed across available blocks on disk surfaces dedicated to the given partition



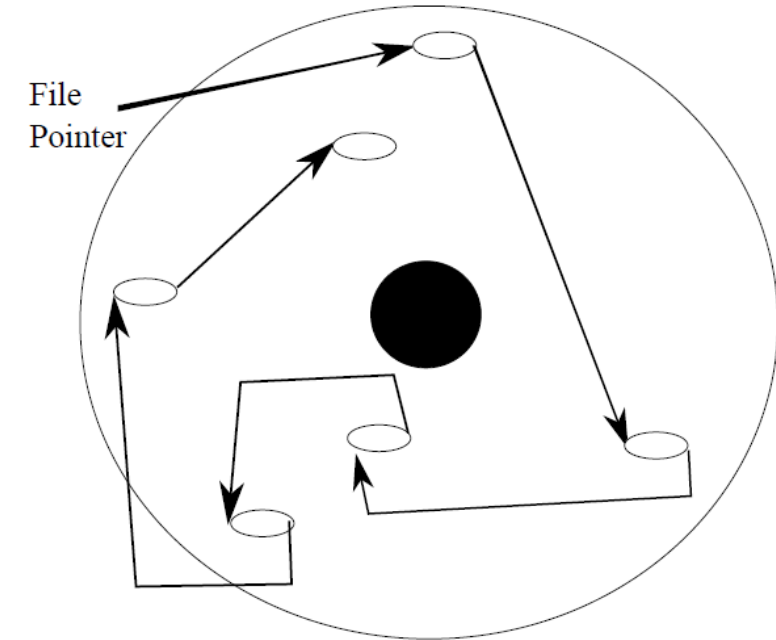
File Access

- File blocks not stored consecutively on a disk
 - makes access more efficient
 - reduces chance of fragmentation
 - creates a linked list structure



Disk blocks store same amount of content; last block may contain a fragment

Disk file stored across disk surface



File Allocation Table (FAT)
loaded into memory to
allow quicker search for
file block

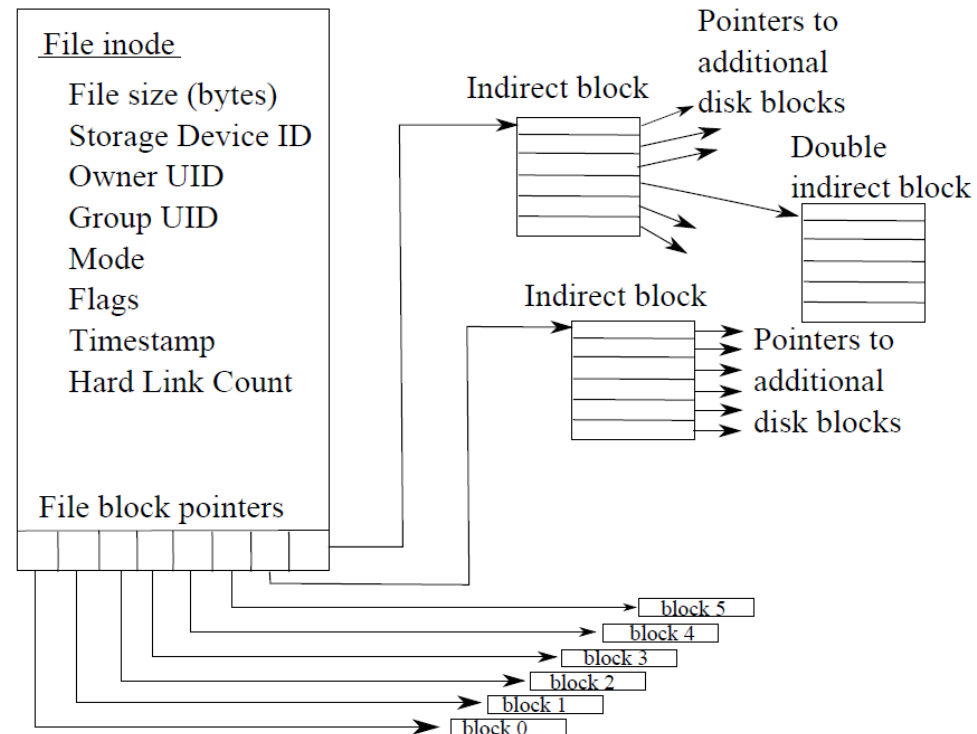
File Allocation Table (portion)

Block	150	151	152	153	154	155	156
Next location	381	153	Bad	156	155	732	EOF



Linux inodes

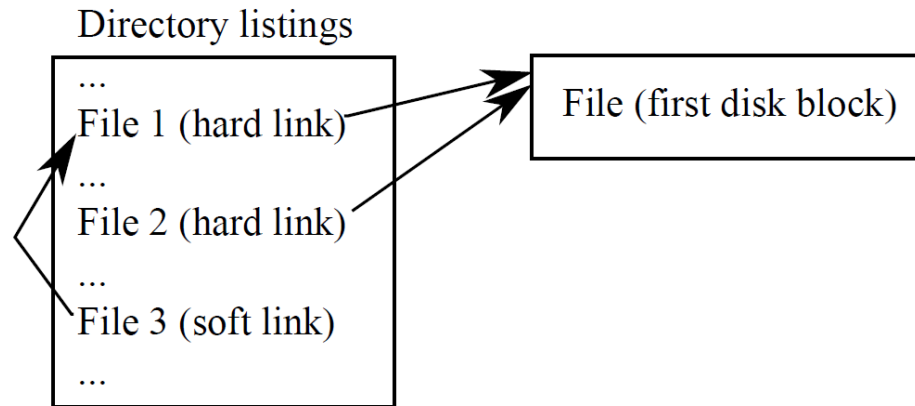
- Each Linux file and directory is represented by an inode (a data structure stored on disk)
 - inode stores information about the file/directory
 - pointers point to disk blocks





Links

- The pointer to the initial disk block is known as a hard link
 - You may create additional links
 - hard links also point at the file
 - soft links point at the initial file name with its hard link
 - In Linux, soft links are called symbolic links
 - In Windows, soft links are called shortcut icons



Links create aliases (alternate names to the item)

Aliases can be dangerous
– if you delete the original file,
the alias points at nothing



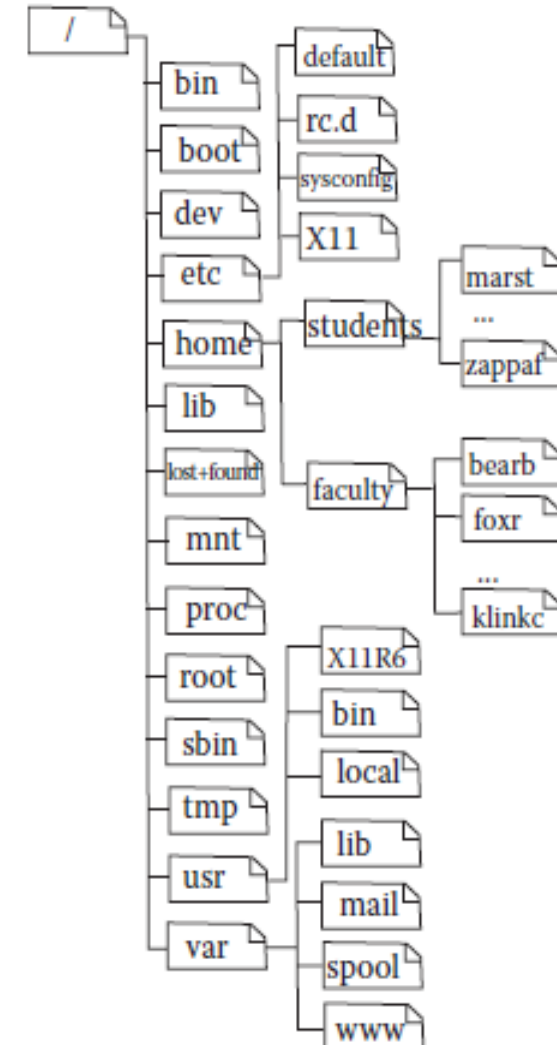
Linux File System

- Top level directories
 - /bin – common user applications and file system operations like cd, ls, rm, cp, mv (see also /usr)
 - /sbin – common system administrator programs
 - /etc – configuration files, initialization scripts, service rule files
 - /root – home directory for root
 - /lib – shared program files
 - /dev – device files – supports program and user interfaces with hardware devices as if they were files
 - /tmp – temporary storage for running applications
 - /boot – boot loader files and Linux kernel stored here
 - /usr – system and applications software (/usr/bin, /usr/sbin)
 - /var – running program data files (log files, printer spool files, error files, email, web space files, locked files)
 - /home – user directories
 - /proc – although this acts like a directory with subdirectories and files like any other directory, it is really stored in memory and consists of information data regarding running processes



The Linux Directory Structure

- As shown here, the top level directories are already created when you install Linux
- Typical partitions divide this space up as follows:
 - / - root partition contains /bin, /root, /sbin, /etc
 - /boot (if not on a separate partition, then this is placed under /)
 - /var – for the /var directory
 - /home – for all user directories
 - swap – the swap space
 - possibly also /usr and /tmp





Windows File System

- Partitions are provided automatically
 - Each partition is lettered
 - A:/B: - floppy disks
 - C: - the hard disk
 - D: - the optical disk
 - E: - mounted flash drive or remote hard disk
 - You can partition your internal hard disk into multiple partitions if desired
- Top level directories already established
 - Windows – the operating system
 - Users – user space
 - Temp – temporary storage space for running applications
 - Program Files – application software



Users

- We require that computer systems have user accounts to ensure
 - Security – only qualified people can use the system
 - Protection – users who create files can establish if and what forms of access other users might have on them
 - Performance – user accounts allow us to gauge the impact that the users are having on the system in order to recommend upgrades
 - Accounting – to illustrate the size and scope of the system and keep track of such things as how many users are using the system at any given time
- Users are authorized through some sort of log in process
 - User name + password is the most common form
 - Some newer or highly secure systems might use keycards or biometric forms of authentication
- Once a user has been authorized, the operating system is able to match the user against access rights (permissions)



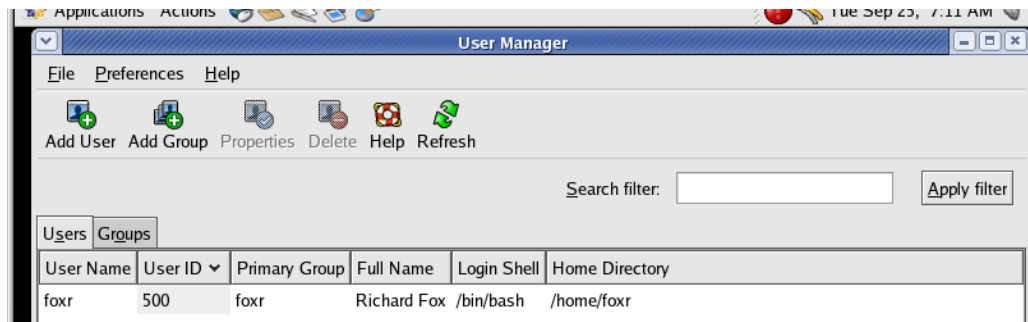
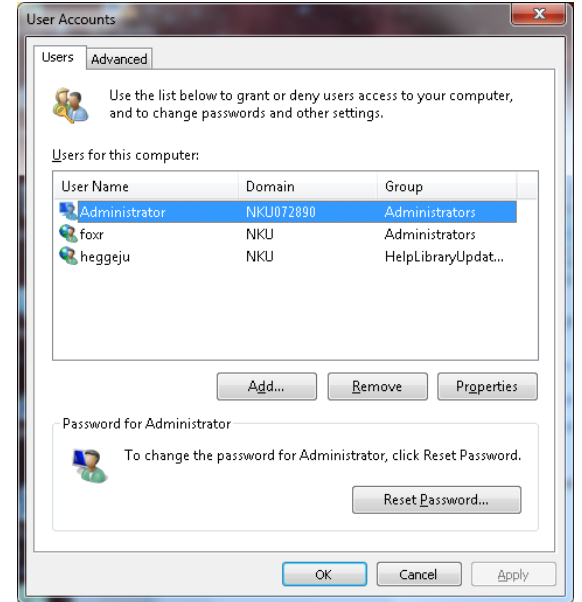
Types of Users

- Generally, we differentiate between two classes of users: administrator and normal user
 - There might be different levels of administrators
 - e.g., some users may be able to mount CD-ROM, others might have access to a printer, etc.
 - The system administrator, or root in Linux/Unix has access to all files, all resources, all programs
- Aside from end-user accounts, software is also given user accounts
 - This is necessary as a program, when run, is given your access rights, but a program might need access to its own files, so the program has its own account
 - In such accounts, there may be no home directory, and usually the software accounts have no log in shell



Creating User Accounts

- In both Windows and Linux there is a simple GUI approach to creating user accounts
 - See the figures below
 - You have to be administrator/root to do this
- There is also a command line way to set up accounts, in Linux, it is useradd
 - `useradd [-u uid [-o]] [-g group] [-G group group group ...] [-d home] [-s shell] [-m] username`





Account Data

- In Linux, you are able to specify a number of different pieces of user data
 - Through both the GUI and command line although more data can be specified using the command line
 - ID number defaults to be 1 greater than the last account
 - username
 - comment field (used for the user's full name)
 - whether a private group should be created for this user or not, and if so, what name to give it (the default is the same name as the username)
 - other groups to add the user to initially
 - what to set as the initial user password
 - log in shell and home directory and whether to create a home directory or not



Groups

- Groups are created so that files can be shared among users in the same group
 - Without groups, the only access rights would be for the owner of the file and everyone else
 - With groups, files can be shared
- Groups are available in both Linux and Windows
 - In Linux, a group is created for every user, known as that user's private group
- Groups can be created via GUI or command line (Linux) and users can be added to groups at any time
 - The groupadd command creates groups and allows you to insert users into groups, or you can use the GUI from the previous slide to modify groups, or you can edit the groups file itself



/etc/passwd

- Stores user account information in Linux
 - Named passwd but it no longer stores passwords because it was viewed as a security hole
 - in place of the password, there is an x
 - passwords now stored in the file /etc/shadow
 - Entry for every user
 - Although you can edit this file as root, it is better to modify user account information using the usermod instruction

The first entry is root, this is followed by various software accounts and finally user accounts (user numbers 500 and 501 here)

Related files are /etc/group and /etc/shadow

```
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/sbin/nologin
news:x:9:13:news:/etc/news:
uucp:x:10:14:uucp:/var/spool/uucp:/sbin/nologin
gdm:x:42:42::/var/gdm:/sbin/nologin
foxr:x:500:500:rf:/home/foxr:/bin/bash
zappaf:x:501:501::/home/zappaf:/bin/bash
```



Linux User Management Instructions

- `passwd` – change password
 - root can change any user's password by doing `passwd username`
 - root can establish when users must change passwords
 - `-f` forces the user to change their password at the next login
 - `-x` allows you to set the maximum number of days until a password expires (needs to be changed)
 - `-w` gives them a warning some number of days before their password expires
 - `passwd -w 7 -30 foxr` – foxr's password must be changed within 30 days with a warning given 7 days prior to the deadline
- `chown` – change ownership of a file
 - format: `chown newuser filename`
- `chgrp` – change the group that owns the file
 - format: `chgrp newgroup filename`
 - `chown newuser:newgroup filename` – changes owner and group
- `usermod` – change a user's data



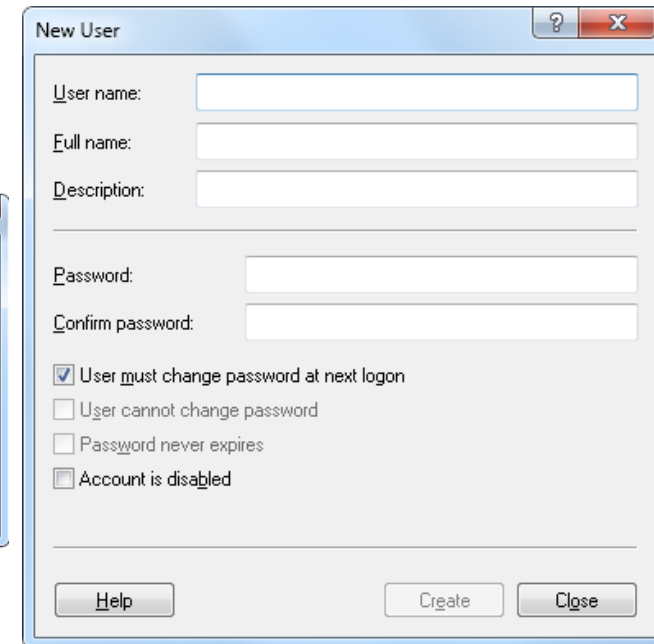
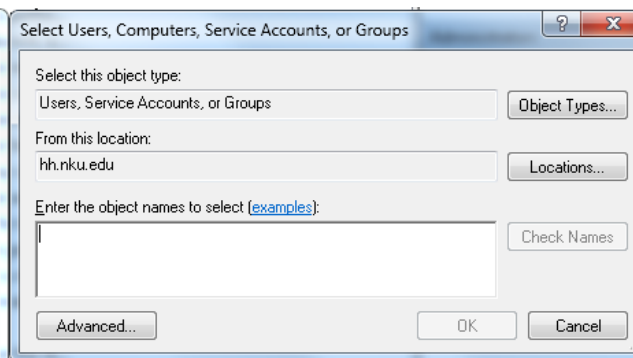
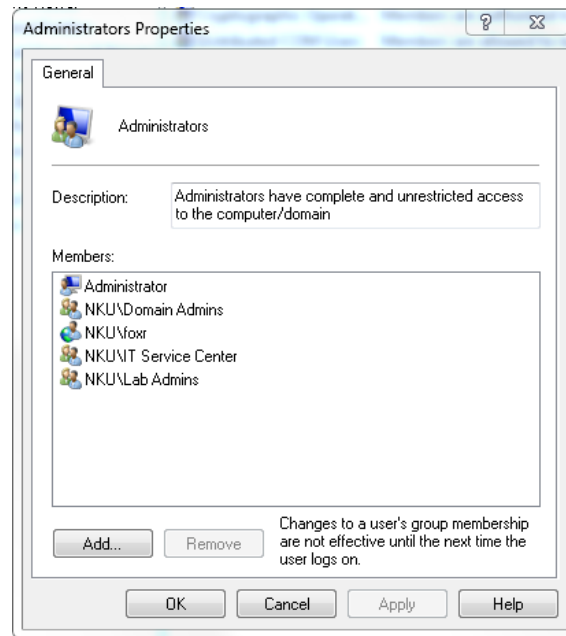
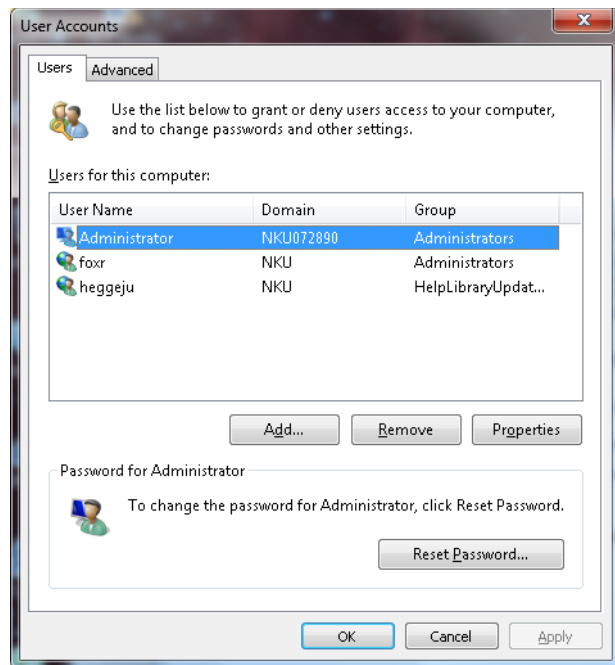
Strong Passwords

- Because of the need to secure our computer systems (due to the confidential nature of data stored there) and the fact that there are a lot of people trying to break into accounts (known as hackers or more properly crackers)
 - We need to ensure that passwords are strong enough to not be easily broken
 - Strong passwords are typically at least 8 characters in length with a combination of letters and non-letters (and possibly both upper and lower case letters), the words do not appear in a dictionary, and need to be changed fairly often (1-3 months)
 - We may also demand that the password not be similar to any previously used password, for instance changing abcdef12 to abcdef13



Windows

- Account creation, group creation, account management, group management and changing passwords is all done through GUI windows





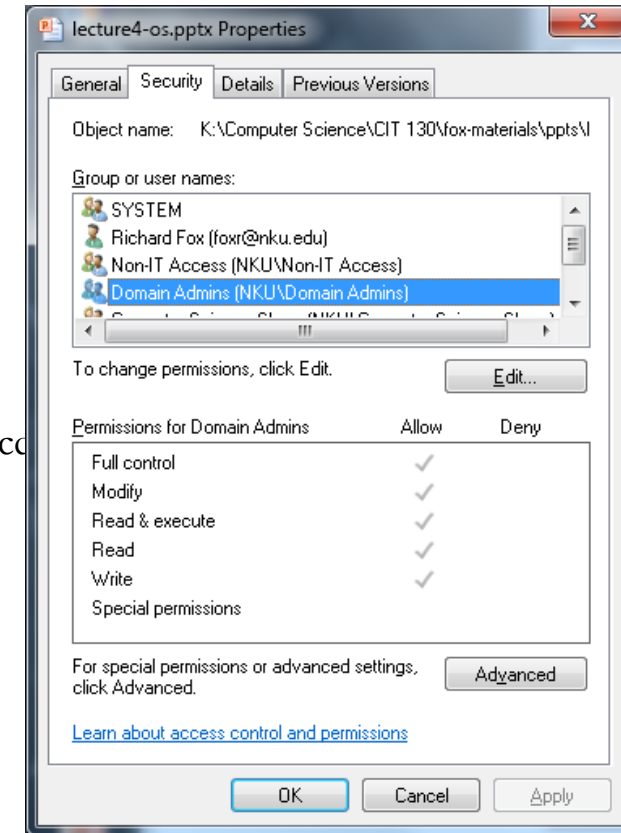
Access Control Lists

- In order to control who can access a resource, resources will need some mechanism for protection
- Some operating systems use ACLs
 - Every file and directory will have its own access control list
 - This list describes what users can access the item and what type of access
 - the advantage of such a list is that every resource can have different access rights for each user
 - the disadvantages are that the length of such lists could take up quite a bit of storage, and they may not be easily altered when, for instance, a new user is added to the system
 - Windows uses a variation of ACLs while Linux uses a simplified set of permissions



Windows Permissions

- Permissions are placed on both files and directories
 - Permissions dictate who can do what to that file or directory
 - Read access – view the contents
 - Write access – save the file, save something into a directory
 - Modify access – save over a previous version
 - Execute access – run the file (only valid if the file is an executable or a script), control
 - Windows also has read & execute combined as well as “full control”



In Windows, use
properties window



Permissions in Linux

- There are three sets of permissions for any resource: owner (user or u), group (g), world (other or o) each has read, write, execute access (r,w, x)
- Change permission use chmod
 - u/g/o+/-r/w/x or ugo=rwx or use 3-digit number
 - Examples:
 - chmod u+x,g-w file1.txt
 - chmod u=rwx,g=rx,o=x file2.txt
- To view a file's permissions, use ls -l

```
total 10
drwxr-xr-x 2 foxr foxr 1024 Aug  5 08:04 Desktop
-rw----- 1 foxr foxr  51 Aug  8 10:32 foo2.txt
-rw----- 1 foxr foxr  46 Aug  8 10:26 foo2.txt~
-rw-rw-r-- 1 foxr foxr  41 Aug 10 12:09 foo3.txt
-rw----- 1 foxr foxr  46 Aug  8 10:26 foo.txt
-rw----- 1 foxr foxr   0 Aug  8 10:25 foo.txt~
```



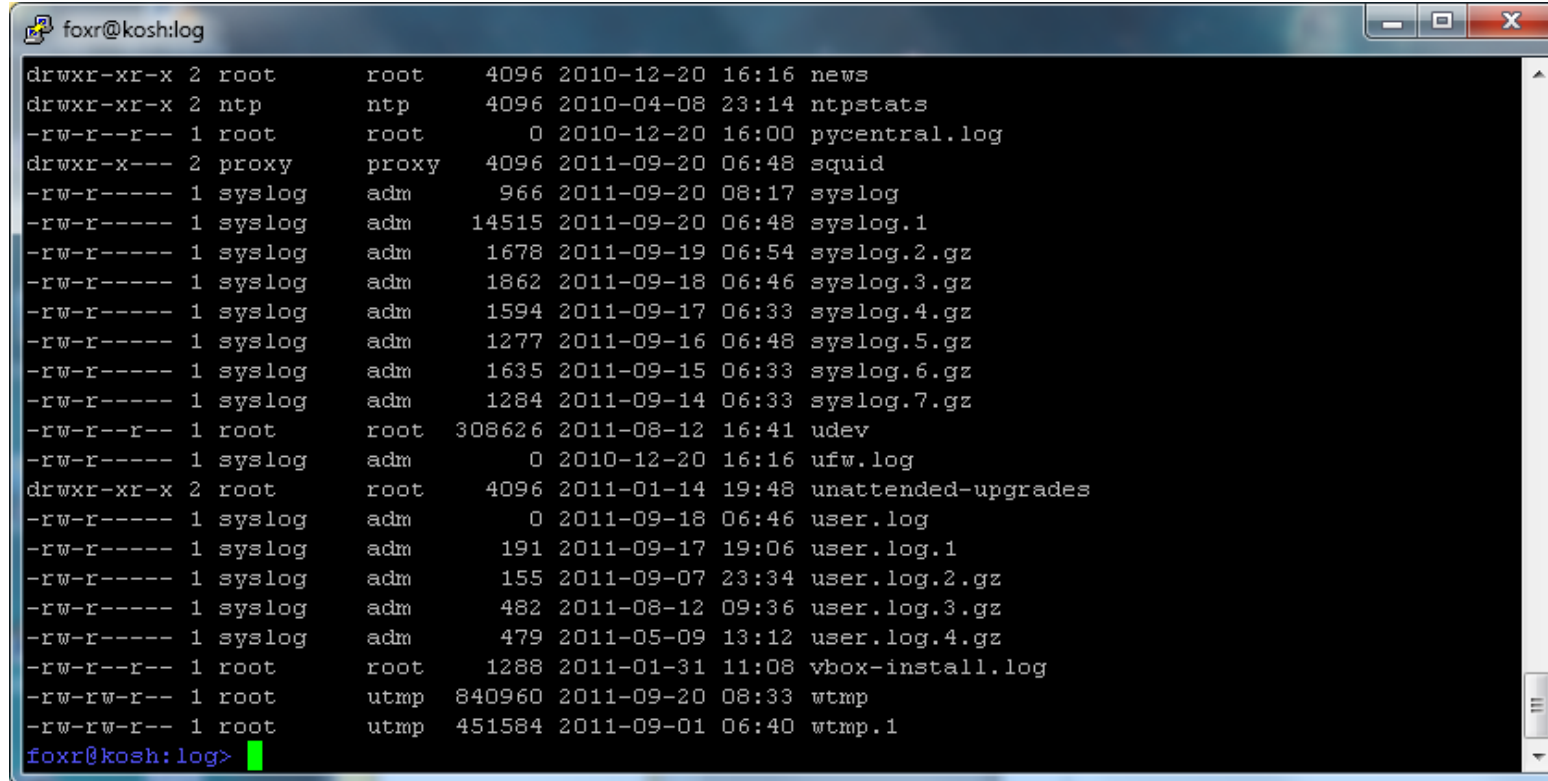
3-Digit Approach

- Add up the 3 values below based on whether you want r, w or x
 - 4 if readable
 - 2 if writable
 - 1 if executable
- file foo3.txt (previous slide) has permissions of rw-rw-r--
 - read + write = $4 + 2 = 6$
 - read = 4
 - 3-digit value = 664
- Example command
 - `chmod 755 file1.txt`
 - owner gets rwx, group and world get r-x (read and execute, not write)
- Other common numbers are:
 - 400 – read-only for owner
 - 444 – readable by all
 - 644 – readable by all, read/write by owner
 - 755 – all access for owner, read/execute for group and world
 - 660 – read/write by owner and group, none to world
 - 000 – no access to anyone (except root)



Viewing a Linux Directory

- Let's examine the excerpt from a Linux directory to see if we can understand the information
 - Can you make sense of the permissions, owner and group entries?



```
foxr@kosh:log
drwxr-xr-x 2 root    root    4096 2010-12-20 16:16 news
drwxr-xr-x 2 ntp     ntp     4096 2010-04-08 23:14 ntpstats
-rw-r--r-- 1 root    root     0 2010-12-20 16:00 pycentral.log
drwxr-x--- 2 proxy   proxy   4096 2011-09-20 06:48 squid
-rw-r----- 1 syslog adm     966 2011-09-20 08:17 syslog
-rw-r----- 1 syslog adm   14515 2011-09-20 06:48 syslog.1
-rw-r----- 1 syslog adm   1678 2011-09-19 06:54 syslog.2.gz
-rw-r----- 1 syslog adm   1862 2011-09-18 06:46 syslog.3.gz
-rw-r----- 1 syslog adm   1594 2011-09-17 06:33 syslog.4.gz
-rw-r----- 1 syslog adm   1277 2011-09-16 06:48 syslog.5.gz
-rw-r----- 1 syslog adm   1635 2011-09-15 06:33 syslog.6.gz
-rw-r----- 1 syslog adm   1284 2011-09-14 06:33 syslog.7.gz
-rw-r--r-- 1 root    root  308626 2011-08-12 16:41 udev
-rw-r----- 1 syslog adm      0 2010-12-20 16:16 ufw.log
drwxr-xr-x 2 root    root    4096 2011-01-14 19:48 unattended-upgrades
-rw-r----- 1 syslog adm      0 2011-09-18 06:46 user.log
-rw-r----- 1 syslog adm    191 2011-09-17 19:06 user.log.1
-rw-r----- 1 syslog adm    155 2011-09-07 23:34 user.log.2.gz
-rw-r----- 1 syslog adm    482 2011-08-12 09:36 user.log.3.gz
-rw-r----- 1 syslog adm    479 2011-05-09 13:12 user.log.4.gz
-rw-r--r-- 1 root    root   1288 2011-01-31 11:08 vbox-install.log
-rw-rw-r-- 1 root    utmp  840960 2011-09-20 08:33 wtmp
-rw-rw-r-- 1 root    utmp  451584 2011-09-01 06:40 wtmp.1
foxr@kosh:log>
```



Switching Users

- In Windows, if you want to switch to another user account, you use the “switch user” facility through the log out/shutdown mechanism
 - This prevents you from having to log out, have the new user log in, do their work, log out and then you having to log in again
- In Linux, from a terminal window, use the su command (switch user)
 - su username
- You will have to log in as the new user but once you are logged in, you have that user’s full access rights
 - Note that su will log in you to that user, but not that user’s home directory, so you might follow this with cd ~
 - Type exit to exit that user’s session and resume yours
 - su <enter> allows you to switch to root
 - If you are root, su username does not require that you specify the user’s password



Sudo Access

- In Linux, the system administrator can provide additional access rights to users through the sudo command
 - A file, /etc/sudoers, lists users and the commands that they can execute
 - entries are listed as: user(s) location=command
 - location is either ALL (all computers in this domain), localhost (this computer only) or a list of specific computers
 - command is the command to be made available, listed with its full path as in /usr/sbin/useradd
 - The user can then run the command using sudo as in
 - `sudo /usr/sbin/useradd -m newuser`
 - With sudo, you can have more access levels between the user and root
 - using sudo, we might permit some (or all) users to perform the shutdown command – why?